Le Potager Solarpunk

Fiche de préparation à l'oral · Projet 2 Unity · Kerouann Touami-Causse · BUT2 Informatique

1. Le jeu en 30 secondes

Le joueur pilote un drone horticole au-dessus d'un potager urbain. Il plante des graines de blé sur des parcelles de terre, les arrose pour accélérer leur croissance, et doit les récolter au bon moment : trop tôt elles ne sont pas prêtes, trop tard elles éclatent. Chaque récolte fait monter le score ; la partie se termine quand l'objectif de récolte est atteint, et le meilleur score est conservé d'une session à l'autre.

2. Commandes

Action	Clavier / souris	Manette
Se déplacer	ZQSD / flèches	Stick gauche
Tourner / incliner la vue	Souris	Stick droit
Monter / descendre	Espace / Ctrl gauche	RB / LB
Planter	Clic gauche	Gâchette droite (RT)
Arroser	Clic droit (maintenu)	Gâchette gauche (LT)

Toutes les commandes passent par un seul asset **Input Actions** : chaque action (Move, Plant...) regroupe les touches clavier ET les boutons manette. Le code lit l'action, jamais une touche précise.

3. Architecture : un script = une responsabilité

Script	Posé sur	Responsabilité
DroneController	Drone	Déplacer le drone (relatif à la caméra), gérer l'altitude, rester dans le potager
CameraOrbit	Main Camera	Suivre le drone et tourner autour (souris / stick droit)
SeedPlanter	Drone	Planter une graine sous le drone si la place le permet
SoilPlot	Chaque parcelle	Marquer la terre cultivable, limiter le nombre de plantes
Plant	Prefab de la plante	Croissance graine → mûr → trop mûr → éclatement

Organisation : tout le contenu du jeu est dans **Assets/_Project**, rangé par type (Scripts, Prefabs, Materials, Scenes, Input, Audio, VFX, UI, Models, Animations). Tous les réglages (vitesses, durées, limites, prefabs) sont des champs **[SerializeField]** modifiables dans l'Inspector, avec une infobulle (Tooltip).

4. Concepts techniques : questions probables et réponses

Déplacement et commandes (jalon 1)

Q : Comment le même code marche-t-il au clavier et à la manette ?

Le script reçoit une `InputActionReference` (l'action *Move*) et lit `ReadValue<Vector2>()`. C'est l'Input System qui fusionne ZQSD, flèches et stick dans une seule valeur entre -1 et 1. Aucun script différent par périphérique.

Q : Pourquoi activer les actions dans OnEnable et les désactiver dans OnDisable ?

C'est le cycle de vie d'un composant Unity : l'action n'écoute les touches que lorsque le composant est actif. On se désabonne aussi des événements dans OnDisable, sinon une action pourrait appeler un objet détruit.

Q : Pourquoi multiplier par Time.deltaTime ?
Update est appelé une fois par image. Sans deltaTime, un PC à 144 FPS déplacerait le drone 2,4 fois plus vite qu'un PC à 60 FPS. Avec deltaTime, la vitesse s'exprime en unités par seconde.

Q : Pourquoi la souris n'utilise-t-elle PAS Time.deltaTime ?
La souris renvoie un déplacement en pixels depuis la dernière image : c'est déjà une quantité par image. Le stick renvoie une position (-1 à 1) qu'on transforme en vitesse : lui doit être multiplié par deltaTime.

Q : Pourquoi la caméra est-elle mise à jour dans LateUpdate ?
LateUpdate s'exécute après tous les Update. Le drone a donc déjà bougé quand la caméra se place : elle suit sa position finale, sans tremblement.

Q : Comment le drone reste-t-il dans le potager et au-dessus du sol ?
Après chaque déplacement, `Mathf.Clamp` borne X et Z entre les limites du terrain, et l'altitude entre une hauteur minimale (1 m) et maximale. Le drone n'a pas de collision physique avec le sol : il vole, on impose directement sa position.

Q : Pourquoi le drone est-il un objet vide avec un visuel en enfant ?
Le code déplace le parent ; les animations (jalon 4) agiront sur l'enfant. Ainsi animation et gameplay ne s'écrasent pas mutuellement (piste de la brique 4.1).

Plantation et croissance (jalon 2)

Q : Comment la graine apparaît-elle exactement sous le drone ?
Un `Physics.Raycast` est lancé vers le bas depuis le drone. La graine est instanciée au point d'impact (`hit.point`) : elle tombe sous le drone, pas devant ni à côté.

Q : Comment sait-on qu'on est au-dessus de la terre ?
Chaque parcelle porte un composant `SoilPlot`. Si le collider touché par le rayon n'en a pas (herbe), on ne plante pas. Un composant est plus sûr qu'un Tag : aucune chaîne de caractères à mal orthographier.

Q : Pourquoi un Prefab ?
La plante est créée des dizaines de fois avec `Instantiate`. Modifier le Prefab modifie toutes les futures plantes.

Q : Comment gérer plusieurs plantes en même temps ?
Une variable simple ne garde qu'un objet ; on stocke donc les plantes dans une `List<Plant>`. Avant de l'utiliser, `RemoveAll(p => p == null)` retire celles qui ont été détruites (récoltées ou éclatées). La liste sert à vérifier l'espacement minimum et le nombre maximum par parcelle.

Q : Comment la plante attend-elle sans bloquer le jeu ?
La croissance est une **coroutine** (`IEnumerator Start()`). À chaque image, elle ajoute `Time.deltaTime × growthSpeed` au temps écoulé, met à jour la taille, puis `yield return null` rend la main au moteur jusqu'à l'image suivante.

Q : Pourquoi un enum GrowthStage plutôt que des booléens ?
Une plante est dans un seul état à la fois : Seed, Ripe ou Overripe. Une seule variable d'état évite les combinaisons impossibles (isSeed et isRipe vrais en même temps). Seul l'état Ripe est récoltable.

Q : Pourquoi deux graines plantées à quelques secondes d'écart ne mûrissent-elles pas ensemble ?
Chaque plante a sa propre coroutine qui démarre à sa plantation, et une petite variation aléatoire (±15 %) est appliquée à ses durées.

Q : Comment la plante disparaît-elle ?
`Destroy(gameObject)` la retire de la scène et de la Hierarchy à la fin de l'étape « trop mûr ».

5. Choix personnels à savoir justifier

- **Caméra orbitale** (souris / stick droit) au lieu d'une caméra fixe : elle démarre en vue de dessus (90°) comme demandé, mais le joueur peut tourner et incliner la vue. Le déplacement reste intuitif car il est relatif à la caméra : « haut » = haut de l'écran.
- **Altitude contrôlable** (Espace / Ctrl, RB / LB). ⚠ La brique 1.1 demande une hauteur constante : justification = le drone ne s'enfonce jamais (plancher à 1 m, plafond à 8 m) et la hauteur reste bornée et maîtrisée. Point de vigilance pour la récolte : la zone de détection des légumes doit monter jusqu'au plafond.
- **Parcelles** : on ne plante que sur la terre, avec un nombre maximum de plantes par parcelle et un espacement minimum, pour un potager lisible et une vraie gestion de l'espace.
- **Thème** : le blé, avec trois visuels distincts (graine, blé mûr, blé trop mûr).
- **Prévu** : plusieurs types de plantes sélectionnables avec affichage à l'écran (molette / croix directionnelle) → défi bonus « Légumes inégaux ».

6. Avancement

Jalon	Contenu	État
1	Le drone survole le potager	Terminé
2	Planter (2.1) · Pousser (2.2) · Arroser (2.3)	2.1 et 2.2 terminées · 2.3 à faire
3	Récolte (trigger) · Score à l'écran	À faire
4	Animations · Sons et particules · Décor solarpunk	À faire
5	Écran titre / fin / pause · Meilleur score (PlayerPrefs)	À faire
6	Rangement · Debug · Build et itch.io · Dépôt Git	Rangement commencé · .gitignore prêt

À mettre à jour au fil du projet.

7. Crédits des assets

Asset	Auteur	Licence	Utilisé pour
À compléter			

Les crédits seront complétés à partir des fichiers de licence des packs téléchargés. Ils doivent aussi figurer sur la page itch.io (brique 6.3).

8. Déroulé conseillé de la démonstration

- Montrer le projet : dossier **_Project** rangé, Hierarchy lisible, Console vide.
- Dans l'Inspector du Drone : vitesse, altitude min/max ; sur le Prefab Plant : les durées de croissance (moins de 30 s pour les trouver).
- Play Mode : déplacement clavier puis manette, rotation de la caméra, montée / descente bloquée à 1 m.
- Planter sur une parcelle, essayer sur l'herbe (refus), essayer trop près (refus), remplir une parcelle (refus au-delà du maximum).
- Laisser une plante passer par les 3 étapes et éclater : elle disparaît de la Hierarchy.
- Suite (quand ce sera fait) : arrosage, récolte, score, écrans, build itch.io.